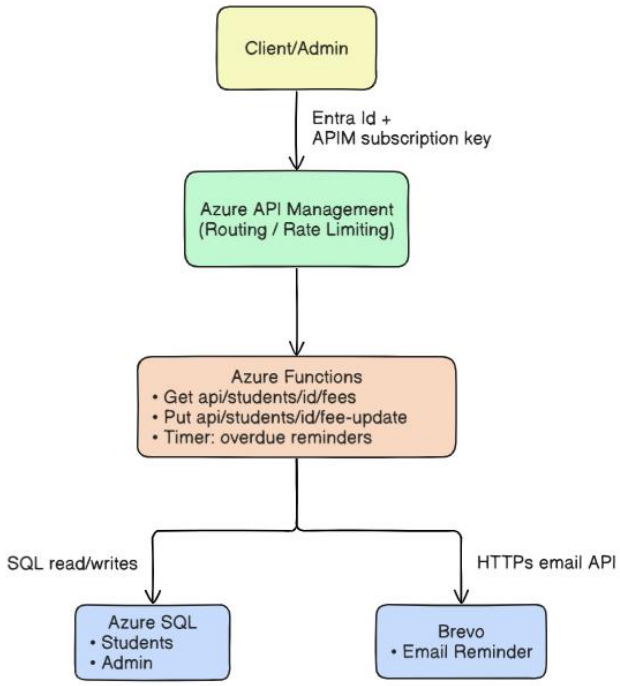
Solution Architecture & Deployment Guide

Azure Fee Management System

1. Architecture



2. Main Components and Request Flow

Component	Responsibility
Azure SQL Database	Stores student fees and administrator records.
Azure Functions	GET /students/{studentId}/fees; protected PUT /students/{studentId}/fee-update; scheduled overdue-fee processing.
API Management	Public API entry point; subscription-key validation and rate limiting according to deployed policy.
Microsoft Entra ID	Authenticates users; the update function checks whether the caller is an authorized administrator.
Brevo + Application Insights	Brevo sends reminders; Application Insights records telemetry and errors.

Repository: https://github.com/7Aayush11/feesystem-azure.git

Deployment note: The GET function currently retrieves a record by StudentID; student-to-record ownership enforcement is a recommended production enhancement if student self-service access is required.

3. Step-by-Step Deployment Guide

Prerequisites: Azure subscription, Azure CLI, Azure Functions Core Tools, Node.js version matching the Function App runtime, Git, access to the configured Azure SQL Database, and a verified Brevo sender/API key.

1. Clone the source - Clone the repository using the **GitHub URL** above, open the project folder, and install dependencies with **npm install**.

2. Configure SQL - Create/verify the Azure SQL Database and the dbo.Students and dbo.Administrators tables. Apply the schema/seed scripts from the repository. Confirm the connection settings and Entra/managed-identity permissions; do not put secrets in source control.

3. Configure the Function App - In Azure Portal, open func-feesystem-dev-ci-app → Configuration. Set required non-secret application settings and email settings: BREVO_API_KEY, BREVO_SENDER_EMAIL, and BREVO_SENDER_NAME. Keep the API key secret, then save and restart the app.

4. Deploy Azure Functions - From the project root (where host.json and package.json are located), sign in with az login and deploy using Azure Functions Core Tools: func azure functionapp publish func-feesystem-dev-ci-app. Check that all expected functions are discovered after deployment.

5. Configure Microsoft Entra authentication - Verify Function App Authentication/Easy Auth and the app registration/token audience settings. Confirm the administrator identity is mapped in dbo.Administrators and that the update function rejects callers who are not authorized administrators.

6. Configure API Management - In apim-feesystem-dev-ci, verify the API base path and GET/PUT operations, backend Function App URL, subscription requirement, and deployed inbound rate-limit policy. Use the APIM endpoint for client requests.

7. Configure the reminder timer - Verify the timer trigger schedule is the intended production schedule (0 30 3 * * * = 09:00 IST for a UTC schedule). Run a controlled test with an overdue test record and verify the function logs and Brevo transactional activity.

8. Validate and monitor - Test GET for an existing, invalid, and missing StudentID; test authorized and unauthorized PUT calls; confirm APIM returns 429 when its configured limit is exceeded. Review Application Insights for requests, exceptions, dependencies, and timer logs.

4. Deployment Verification Checklist

Check	Expected evidence
Database	Tables exist; StudentID is uniquely indexed; row count matches the assignment requirement (at-least 20 records).
Functions	GET, PUT and overdue reminder functions appear after deployment; no startup/indexing errors.
Security / APIM	Entra token works; non-admin PUT is rejected; subscription key and rate policy are verified.
Email / monitoring	Test reminder is visible in provider logs/inbox; Application Insights shows execution telemetry.
Scalability	System works fast, cold start in initial startup on free resources.